

深圳大学实验报告

课程名称: 数值计算方法

实验项目名称: Newton 插值算法性能对比

学院: 计算机与软件学院

专业: 人工智能卓越班

指导教师: 吴晓群、卯丙

报告人: 邓瑞霖 学号: 2024150040 班级: 01

实验时间: 2026 年 5 月 7 日-2026 年 5 月 31 日

实验报告提交时间: 2026 年 5 月 15 日星期五

教务部制

实验目的与要求:

- 1.掌握 Larange 插值方法的优缺点及其改进思路, 以及 Newton 插值方法的基本思想与基本算法
- 2.掌握差商与差分等重要概念, 并编程计算函数的不同阶差商、差分值并以表格形式输出
- 3.实现 Newton 插值多项式中基函数的可视化展示
- 4.编程实现 Newton 插值算法

基本原理与算法 (详细阐述实验涉及数值计算方法的数学原理及其对应的算法步骤)

1. Larange 插值方法的优缺点及其改进思路

用 Lagrange 插值计算函数的近似值, 若对 $n+1$ 个插值节点所得到的计算结果的精度不满意而需要增加新的插值节点时, Lagrange 插值的每一项都需重新计算, 这很不经济, 改善这种算法的办法之一是逐步线性插值方法, 也就是将高次插值问题用逐次线性插值的算法来实现.

设已给区间 $[a, b]$ 上 $n+1$ 个不同插值节点 $x_0, x_1, \dots, x_n$ 的函数值为 $y_0, y_1, \dots, y_n$, 用

$i_k (k = 0, 1, 2, \dots, n)$ 表示一个非负的整数序列, 把由 $k+1$ 个节点 $x_{i_0}, x_{i_1}, \dots, x_{i_k}$ 所确定的次数不超过 k 的插值多项式记作

$$P_{x_{i_0}, x_{i_1}, \dots, x_{i_k}}(x)$$

特别的, 零次多项式记作 $P_{i_r} = y_{i_r} (r = 0, 1, 2, \dots, k)$, 于是

$$\begin{aligned} P_{x_{i_0}, x_{i_1}, \dots, x_{i_k}}(x) &= \frac{x - x_0}{x_{i_k} - x_{i_0}} P_{x_{i_1}, \dots, x_{i_k}}(x) + \frac{x - x_{i_k}}{x_{i_0} - x_{i_j}} P_{x_{i_0}, \dots, x_{i_{k-1}}}(x) \\ &= \frac{1}{x_{i_k} - x_{i_0}} \begin{vmatrix} P_{x_{i_0}, \dots, x_{i_{k-1}}}(x) & x_{i_0} - x \\ P_{x_{i_1}, \dots, x_{i_k}}(x) & x_{i_k} - x \end{vmatrix} \end{aligned}$$

如果已经算出 $P_{x_{i_1}, \dots, x_{i_k}}(x)$ 及 $P_{x_{i_0}, x_{i_1}, \dots, x_{i_{k-1}}}(x)$ 在 x 点处的值, 经过一次线性插值即可

算出 $P_{x_{i_0}, x_{i_1}, \dots, x_{i_k}}(x)$ 的值, 这样由 $x_{i_0}, x_{i_1}, \dots, x_{i_k}$ 确定的高次插值问题可以逐步转化为线性插值的算法来解决.

1.1 Aitken 和 Neville 插值

已知被插值函数 $f(x)$ 的三个节点及其函数值 $(x_0, f(x_0))$ 、 $(x_1, f(x_1))$ 和 $(x_2, f(x_2))$ 。先过两个点 $(x_0, f(x_0))$ 和 $(x_1, f(x_1))$ 构造直线（一次插值）

$$P_{01} = \frac{x - x_1}{x_0 - x_1} f(x_0) + \frac{x - x_0}{x_1 - x_0} f(x_1)$$

再过两点 $(x_0, f(x_0))$ 和 $(x_2, f(x_2))$ 构造直线（一次插值）

$$P_{02} = \frac{x - x_2}{x_0 - x_2} f(x_0) + \frac{x - x_0}{x_2 - x_0} f(x_2)$$

当插值点 x 给定， P_{01}, P_{02} 都是函数值，因此再这个意义上将 (x_1, P_{01}) 和 (x_2, P_{02}) 也看成是两个点，对这两个点再进行线性插值，有

$$P_{012} = \frac{x - x_2}{x_1 - x_2} P_{01} + \frac{x - x_1}{x_2 - x_1} P_{02}$$

并化简可得：

$$P_{012} = \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} f(x_0) + \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} f(x_1) + \frac{(x - x_1)(x - x_0)}{(x_2 - x_1)(x_2 - x_0)} f(x_2)$$

Aitken 逐步线性插值方法：

x_0	y_0		$x_0 - x$
x_1	y_1	$P_{01}(x)$	$x_1 - x$
x_2	y_2	$P_{02}(x) \quad P_{012}(x)$	$x_2 - x$
x_3	y_3	$P_{03}(x) \quad P_{013}(x) \quad P_{0123}(x)$	$x_3 - x$

Neville 逐步线性插值方法：

x_0	y_0		$x_0 - x$
x_1	y_1	$P_{01}(x)$	$x_1 - x$
x_2	y_2	$P_{12}(x) \quad P_{012}(x)$	$x_2 - x$
x_3	y_3	$P_{23}(x) \quad P_{123}(x) \quad P_{0123}(x)$	$x_3 - x$

1.2 Lagrange 插值缺点

不具有承袭性：每当增加一个节点时，不仅要增加求和的项数，而且以前的各项也必须重新计算。

2. Newton 插值

2.1 差商及其性质

已知函数 $f(x)$ 的 $n+1$ 插值节点为 (x_i, y_i) , $y_i = f(x_i)$, $i = 0, 1, \dots, n$, $\frac{f(x_i) - f(x_j)}{x_i - x_j}$ 称为

$f(x)$ 在点 x_i, x_j 的一阶差商，也称为一阶均差，记为 $f[x_i, x_j]$ ，即

$$f[x_i, x_j] = \frac{f(x_i) - f(x_j)}{x_i - x_j}$$

一阶差商的差商 $\frac{f[x_i, x_j] - f[x_j, x_k]}{x_i - x_k}$ 称为 $f(x)$ 在点 x_i, x_j, x_k 的二阶差商, 也称二阶均差, 记

为 $f[x_i, x_j, x_k]$, 即

$$f[x_i, x_j, x_k] = \frac{f[x_i, x_j] - f[x_j, x_k]}{x_i - x_k}$$

一般地, $k-1$ 阶差商的差商 $\frac{f[x_0, x_1, \dots, x_{k-1}] - f[x_1, x_2, \dots, x_k]}{x_0 - x_k}$ 称为 $f(x)$ 在点 $x_0, x_1, \dots, x_k$ 的 k

阶差商, 记为 $f[x_0, x_1, \dots, x_k]$, 即

$$f[x_0, x_1, \dots, x_k] = \frac{f[x_1, x_1, \dots, x_{k-1}] - f[x_1, x_2, \dots, x_k]}{x_0 - x_k}$$

特别的, 若 $f[x_i] = f(x_i), i = 0, 1, \dots, n$, $f[x_i]$ 称为 $f(x)$ 关于 x_i 的零阶差商

性质 1:

n 阶差商可以表示成 $n+1$ 个函数值 $f(x_0), f(x_1), \dots, f(x_n)$ 的线性组合, 即

$$f[x_0, x_1, \dots, x_n] = \sum_{i=0}^n \frac{f(x_i)}{(x_i - x_0) \dots (x_i - x_{i-1})(x_i - x_{i+1}) \dots (x_i - x_n)}$$

性质 2: 对称性

差商与节点的顺序无关, 如:

$$\begin{aligned} f[x_0, x_1] &= f[x_1, x_0] \\ f[x_0, x_1, x_2] &= f[x_1, x_0, x_2] = f[x_0, x_2, x_1] \end{aligned}$$

性质 3:

若 $f(x)$ 是 x 的 n 次多项式, 则一阶差商 $f[x, x_0]$ 是 x 的 $n-1$ 次多项式, 二阶差商 $f[x, x_0, x_1]$ 是 x 的 $n-2$ 次多项式; 一般地, 函数 $f(x)$ 的 $k(k \leq n)$ 阶差商 $f[x, x_0, \dots, x_{k-1}]$ 是 x 的 $n-k$ 次多项式, 而 $k > n$ 时, k 阶差商为零

2.2 Newton 插值多项式

构造多项式:

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots + a_n(x - x_0) \dots (x - x_{n-1})$$

使其满足: $P_n(x_k) = y_k (k = 0, 1, 2, \dots, n)$

由插值条件: $P_n(x_i) = f_i(x_i) (i = 0, 1, \dots, n)$

当 $x = x_0$ 时, $P_n(x_0) = a_0 = f_0 = f[x_0]$, $a_0 = f[x_0]$

当 $x = x_1$ 时, $P_n(x_1) = f_1 = a_0 + a_1(x_1 - x_0) = f_0 + a_1(x_1 - x_0)$, $a_1 = \frac{f_1 - f_0}{x_1 - x_0} =$

$$f[x_0, x_1]$$

当 $x = x_2$ 时, $P_n(x_2) = f_2 = a_0 + a_1(x_2 - x_0) + a_2(x_2 - x_0)(x_2 - x_1)$, $a_2 =$

$$\frac{\frac{f_2 - f_0}{x_2 - x_0} - \frac{f_1 - f_0}{x_1 - x_0}}{x_2 - x_1} = f[x_0, x_1, x_2]$$

依次递推可得:

$$a_k = f[x_0, x_1, \dots, x_k] = \frac{f[x_0, x_1, \dots, x_{k-2}, x_k] - f[x_0, x_1, \dots, x_{k-1}]}{x_k - x_{k-1}}$$

$$(k = 0, 1, \dots, n)$$

构造多项式:

$$N_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots + a_n(x - x_0) \dots (x - x_{n-1})$$

Newton 插值多项式:

$$N_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots$$

$$+ f[x_0, x_1, \dots, x_n](x - x_0)(x - x_1) \dots (x - x_{n-1})$$

$$= \sum_{k=0}^n f[x_0, x_1, \dots, x_k] \omega(\mathbf{X})$$

Newton 插值多项式还可以直接从差商的定义得到, 设 $\forall x \in [a, b]$, 则由 $f(x)$ 的一阶差商定义式得

$$f(x) = f(x_0) + f[x, x_0]$$

由 $f(x)$ 的二阶差商定义式得

$$f[x, x_0] = f[x_0, x_1] + f[x, x_0, x_1]$$

一般地, 由 $f(x)$ 的 $n+1$ 阶差商定义式得

$$f[x, x_0, x_1, \dots, x_{n-1}] = f[x_0, x_1, \dots, x_n] + f[x, x_0, x_1, \dots, x_n](x - x_n)$$

一次将后式代入前式得:

$$f(x_0) + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots$$

$$+ f[x_0, x_1, \dots, x_n](x - x_0)(x - x_1) \dots (x - x_{n-1})$$

$$+ f[x, x_0, x_1, \dots, x_n](x - x_0)(x - x_1) \dots (x - x_n)$$

$$= N_n(x) + R_n(x)$$

牛顿插值余项:

$$R_n(x) = f[x, x_0, \dots, x_n](x - x_0)(x - x_1) \dots (x - x_n)$$

$$= f[x, x_0, \dots, x_n] \omega_{n+1}(x_0, \dots, x_n) = \frac{f^{(n+1)}(\varepsilon)}{(n+1)!} \omega_{n+1}(x)$$

性质 4: 差商与导数的关系:

若 $f(x)$ 在 $[a, b]$ 上存在 n 阶导数, 且 $x_i \in [a, b] (i = 0, 1, 2, \dots, n)$, 则

$$f[x_0, x_1, \dots, x_n] = \frac{f^{(n)}(\varepsilon)}{n!}, \quad \varepsilon \in (a, b)$$

3. 差分及其性质

设函数 $y = f(x)$ 在等距节点 $x_k = x_0 + kh (k = 1, 2, \dots, n)$ 上的函数值 $f_k =$

$f(x_k), h = \frac{b-a}{n}$, 称为步长

定义 1: 差分及算子

引入记号

$$\Delta f_k = f_{k+1} - f_k,$$

$$\nabla f_k = f_k - f_{k-1},$$

$$\delta f_k = f\left(x_k + \frac{1}{2}h\right) - f\left(x_k - \frac{1}{2}h\right) = f_{k+\frac{1}{2}} - f_{k-\frac{1}{2}}$$

分别称为 $f(x)$ 在 x_k 处以 h 为步长向前一阶差分、向前一阶差分和一阶中心差分

符号 Δ, ∇, δ 分别称为向前一阶差分算子, 向前一阶差分算子和中心一阶差分算子

利用一阶差分可以定义二阶差分

$$\Delta^2 f_k = \Delta f_{k+1} - \Delta f_k = f_{k+2} - 2f_{k+1} + f_k$$

一般地可定义 m 阶差分为:

$$\Delta^m f_m = \Delta^{m-1} f_{k+1} = \Delta^{m-1} f_k$$

$$\nabla^m f_k = \nabla^{m-1} f_k - \nabla^{m-1} f_{k-1}$$

$$\delta^m f_k = \delta^{m-1} f_{k+1/2} - \delta^{m-1} f_{k-1/2}$$

再引入下列常用的算子符号

$$I f_k = f_k, \quad E f_k = f_{k+1}$$

并称 I 为恒等算子, E 为位移算子

各算子之间有如下关系:

$$\Delta f_k = f_{k+1} - f_k = E f_k - I f_k = (E - I) f_k$$

故有 $\Delta = E - I$, 同理有

$$\delta = E^{\frac{1}{2}} - E^{-\frac{1}{2}}, \quad \nabla = I - E^{-1}$$

性质 5:

常数的差分等于零。

性质 6:

函数值可以表示各阶差分

$$\Delta^n f_k = (E - I)^n f_k = \sum_{j=0}^n (-1)^j C_n^j E^{n-j} f_k = \sum_{j=0}^n (-1)^j C_n^j f_{n+k-j};$$

$$\nabla^n f_k = (I - E^{-1})^n f_k = \sum_{j=0}^n (-1)^{n-j} C_n^j E^{j-n} f_k = \sum_{j=0}^n (-1)^{n-j} C_n^j f_{k+j-n}.$$

性质 7: 各阶差分可以表示函数值

$$f_{n+k} = E^n f_k = (I + \Delta)^n f_k = \left(\sum_{j=0}^n C_n^j \Delta^j \right) f_k$$

于是

$$f_{n+k} = E^n f_k = \sum_{j=0}^n C_n^j \Delta^j f_k$$

性质 8: 差商与差分的关系

$$f[x_k, x_{k+1}] = \frac{f_{k+1} - f_k}{x_{k+1} - x_k} = \frac{\Delta f_k}{h}$$

$$f[x_k, x_{k+1}, x_{k+2}] = \frac{f[x_{k+1}, x_{k+2}] - f[x_k, x_{k+1}]}{x_{k+2} - x_k} = \frac{1}{2h^2} \Delta^2 f_k$$

一般地, 有

$$f[x_k, x_{k+1}, \dots, x_{k+m}] = \frac{1}{m! h^m} \Delta^m f_k$$

同理, 对向后差分有:

$$f[x_k, x_{k-1}, \dots, x_{k-m}] = \frac{1}{m! h^m} \nabla^m f_k$$

利 用 $f[x_0, x_1, \dots, x_n] = \frac{f^{(n)}(\varepsilon)}{n!}$, $\varepsilon \in (a, b)$ 和 $f[x_k, x_{k+1}, \dots, x_{k+m}] = \frac{1}{m! h^m} \Delta^m f_k$ 得:

$$\Delta^n f_k = h^n f^{(n)}(\varepsilon), \varepsilon \in (x_k, x_{k+n})$$

4. 差分与等距节点的 Newton 插值

在 Newton 插值多项式中为计算差商需进行多次除法运算, 当节点为等距时, 可以用差分来代替差商, 从而减少了计算量。

若节点 $x_k = x_0 + kh (k = 0, 1, \dots, n)$ ，计算 x_0 点附近的函数 $f(x)$ ，令 $x = x_0 + th (0 \leq t \leq 1)$ ，于是 $\omega_{k+1}(x) = \prod_{j=0}^k (x - x_j) = t(t-1)\dots(t-k)h^{k+1}$ 代入 Newton 插值多项式

$$N_n(x) = f(x_0) + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots$$

$$N_n(x_0 + th) = f_0 + t\Delta f_0 + \frac{t(t-1)}{2!}\Delta^2 f_0 + \dots + \frac{t(t-1)\dots(t-n+1)}{n!}\Delta^n f_0$$

称为 Newton 向前插值公式，简称前插公式，其余项由式得

$$R_n(x) = \frac{t(t-1)\dots(t-n)}{(n+1)!}h^{n+1}f^{(n+1)}(\varepsilon), \varepsilon \in (x_0, x_n)$$

如果求函数 $f(x)$ 在 x_n 点附近的函数值 $f(x)$ ，插值点应按 $x_n, x_{n-1}, \dots, x_1$ 的次序排列

$$N_n(x) = f(x_n) + f[x_n, x_{n-1}](x - x_n) + f[x_n, x_{n-1}, x_{n-2}](x - x_n)(x - x_{n-1}) \\ + \dots + f[x_n, x_{n-1}, \dots, x_0](x - x_n)(x - x_{n-1})\dots(x - x_1)$$

作变换 $x = x_n + th (-1 \leq t \leq 0)$ ，得

$$N_n(x_n + th) = f_n + t\nabla f_n + \frac{t(t+1)}{2!}\nabla^2 f_n + \dots + \frac{t(t+1)\dots(t+n-1)}{n!}\Delta^n f_n$$

称为 Newton 向后插值公式，简称后插公式，其余项：

$$R_n(x) = \frac{t(t+1)\dots(t+n)}{(n+1)!}h^{n+1}f^{(n+1)}(\varepsilon), \varepsilon \in (x_0, x_n)$$

通常求插值节点开头部份插值点附近函数值时用 Newton 前插公式，求插值点末尾部份插值点附近函数值时用 Newton 后插公式

实验过程及内容：

1. 问题描述（描述需要解决的具体数值计算问题）

2. 实验步骤：

- (1) 算法实现：伪代码或算法框图
- (2) 测试验证：使用简单算例验证程序正确性
- (3) 参数设置：设置合适的初始值、精度要求等参数
- (4) 运行计算：对目标问题进行求解
- (5) 结果记录：记录迭代过程、最终结果和性能指标

3. 程序设计（具体的 MATLAB 程序）

1. 问题描述

- 1) 编程展示 4 次 Newton 插值多项式中的所有基函数，插值节点自行选取；
- 2) 应用下表所给出的数据，编写一个用 Newton 插值算法近似计算 $f(x)$ 的程序，输出 0 至 4 阶的差商表，且至少选择两组不同的数据节点用 $N_3(x)$ 近似计算 $x=21.4$ 时的 $f(x)$

的值，结果保留 5 为小数，并比较其计算性能，最后画出点集 $\{(x_i, f(x_i)) \mid i = 0, 1, 2, \dots, 4\}$ 和 $N_3(x)$ 的图形，并进行数值比较。

x_i	20	21	22	23	24
$f(x_i)$	1.30103	1.32222	1.34242	1.36173	1.38021

2. 实验步骤

2.1 实验一

编程展示 4 次 Newton 插值多项式中的所有基函数，插值节点自行选取

2.1.1 算法实现

Newton 插值法的基函数分别为：

$$f_0(x)=1$$

$$f_1(x)=(x-x_0)$$

$$f_2(x)=(x-x_0)(x-x_1)$$

$$f_3(x)=(x-x_0)(x-x_1)(x-x_2)$$

$$f_4(x)=(x-x_0)(x-x_1)(x-x_2)(x-x_3)$$

2.1.2 参数设置

选择等距节点 $x_0=0, x_1=1, x_2=2, x_3=3, x_4=4$

2.1.3 运行计算

%% 4 次 Newton 插值基函数演示（确保可运行版）

clear all; clc; close all;

% 1. 定义节点和数据

x_nodes = [0, 1, 2, 3, 4]; % 插值节点

n = length(x_nodes) - 1; % 多项式次数 (n=4)

m = 1000; % 绘图点数

% 2. 生成绘图用的 x 轴数据

x_plot = linspace(-1, 5, m)';

% 3. 初始化基函数矩阵

% phi 的每一列代表一个基函数: [$\phi_0, \phi_1, \phi_2, \phi_3, \phi_4$]

phi = zeros(m, n + 1);

% 4. 计算基函数（核心部分，已修复维度问题）

for k = 1:n+1

if k == 1

% $\phi_0(x) = 1$

phi(:, k) = ones(m, 1);

else

% $\phi_k(x) = \phi_{k-1}(x) * (x - x_{k-2})$

% 注意：这里必须使用点乘 .*

```

        idx = k - 1; % 对应的节点索引 (因为 k 从 1 开始)
        phi(:, k) = phi(:, k-1) .* (x_plot - x_nodes(idx));
    end
end

% 5. 绘制图形
figure('Color', 'white', 'Position', [100, 100, 800, 600]);
hold on; grid on; box on;

% 绘制每个基函数
colors = jet(n + 1);
for i = 1:n+1
    plot(x_plot, phi(:, i), ...
        'Color', colors(i, :), ...
        'LineWidth', 2, ...
        'DisplayName', ['$\phi_{' num2str(i-1) '(x)$']);
end

% 标记节点位置
plot(x_nodes, zeros(size(x_nodes)), 'ko', ...
    'MarkerSize', 8, ...
    'MarkerFaceColor', 'k');

% 图形美化
xlabel('$x$', 'Interpreter', 'latex', 'FontSize', 14);
ylabel('$\phi_i(x)$', 'Interpreter', 'latex', 'FontSize', 14);
title('Newton Interpolation Basis Functions ($n=4$)', ...
    'Interpreter', 'latex', 'FontSize', 16);
legend('Location', 'northwest', 'Interpreter', 'latex', 'FontSize', 12);
set(gca, 'FontSize', 11, 'LineWidth', 1.2);
hold off;

```

2.1.4 结果记录

4 次 Newton 插值多项式中的所有基函数:

$$f_0(x)=1$$

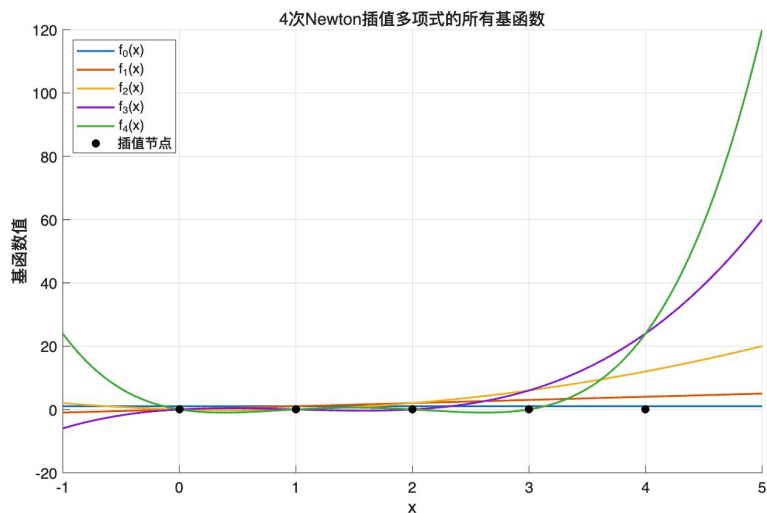
$$f_1(x)=x$$

$$f_2(x)=x(x-1)$$

$$f_3(x)=x(x-1)(x-2)$$

$$f_4(x)=x(x-1)(x-2)(x-3)$$

基函数图像:



2.2 实验二

应用下表所给出的数据，编写一个用 Newton 插值算法近似计算 $f(x)$ 的程序，输出 0 至 4 阶的差商表，且至少选择两组不同的数据节点用 $N_3(x)$ 近似计算 $x=21.4$ 时的 $f(x)$ 的值，结果保留 5 为小数，并比较其计算性能，最后画出点集 $\{(x_i, f(x_i)) \mid i = 0, 1, 2, \dots, 4\}$ 和 $N_3(x)$ 的图形，并进行数值比较。

x_i	20	21	22	23	24
$f(x_i)$	1.30103	1.32222	1.34242	1.36173	1.38021

2.2.1 算法实现

算法伪代码:

```

Input:
    x[1...n]      // 等距插值节点 (按行)
    y[1...n]      // 对应节点函数值
    xi           // 待求插值点的自变量值

Output:
    yi           // 在 xi 处的插值结果

Begin
    // 1. 计算步长与归一化变量 t
    h ← x[2] - x[1]
    t ← (xi - x[1]) / h

    // 2. 构造差分表 Y
    Create matrix Y of size n × n, initialized to 0
    for i ← 1 to n do
        Y[i][1] ← y[i]
    end for

    for k ← 1 to n-1 do

```

```

        for i ← 1 to n-k do
            Y[i][k+1] ← Y[i+1][k] - Y[i][k]
        end for
    end for

    // 3. Newton 向前插值求和
    yi ← Y[1][1]

    for i ← 1 to n-1 do
        term ← 1
        for k ← 0 to i-1 do
            term ← term × (t - k)
        end for
        yi ← yi + Y[1][i+1] × term / factorial(i)
    end for

    return yi
End

```

2.2.2 测试验证

函数为 $f(x)=x+1$ 插入节点为 $x = [1, 2]$, $y=[2, 3]$, 计算插值多项式 $x=1.5$ 的函数值
 运行结果为 $N_1(1.5) = 2.5$
 运行结果与理论结果相同代码正确

2.2.3 参数设置

两组节点选择:

1. $x = [20, 21, 22, 23]$
2. $x = [21, 22, 23, 24]$

2.2.4 运行计算

输出 0-4 阶差商表:

```

disp('0 至 4 阶差商表 (保留五位小数) : ');
x = [20, 21, 22, 23, 24];
y = [1.30103, 1.32222, .134242, 1.36173, 1.38021];
divided_diff = zeros(length(x), length(x));
divided_diff(:, 1) = y';

for j = 2:length(x)
    for i = j:length(x)
        divided_diff(i, j) = (divided_diff(i, j-1) - divided_diff(i-1, j-1)) / ...
            (x(i) - x(i-j+1));
    end
end
end

```

```

% 保留五位小数
divided_diff = round(divided_diff, 5);

disp(array2table(divided_diff, ...
    'VariableNames', arrayfun(@(n) sprintf('第 %d 阶差商 ', n), 0:length(x)-1,
    'UniformOutput', false), ...
    'RowNames', arrayfun(@(n) sprintf('x_%d', n), 0:length(x)-1, 'UniformOutput',
    false)));
for i = 1: length(x)
for j = 1: length(x)
fprintf('%0.5f',divided_diff(i,j));
end
fprintf('\n');
end

errors = 0;
for i = 1:length(x)
errors = errors + abs( newtonint1(x1,y1,x(i)) - y(i));
end
fprintf('第一组节点的  $N_3(x)$  在原始点上的绝对误差: %0.5f\n',errors);
errors = 0;
for i = 1:length(x)
errors = errors + abs( newtonint2(x1,y1,x(i)) - y(i));
end
fprintf('第二组节点的  $N_3(x)$  在原始点上的绝对误差: %0.5f\n',errors);

```

选择两组不同的数据节点用 $N_3(x)$ 近似计算 $x=21.4$ 时的 $f(x)$ 的值:

```

x = [20, 21, 22, 23, 24];
y = [1.30103, 1.32222, 1.34242, 1.36173, 1.38021];
xi = 21.4
% 2. 选择两组不同的数据节点用  $N_3(x)$  近似计算  $x=21.4$  时的  $f(x)$  的值
% 第一组节点:  $x_0, x_1, x_2, x_3$ 
x1 = x(1:4);
y1 = y(1:4);
% 第二组节点:  $x_1, x_2, x_3, x_4$ 
x2 = x(2:5);
y2 = y(2:5);
% 输出计算结果, 保留 5 位小数
fprintf('第一组节点 ( $x_0$ - $x_3$ ) 计算的  $x=21.4$  时  $f(x)$  的值为: %0.5f\n',
newtonint1(x1,y1,21.4));
fprintf('第二组节点 ( $x_1$ - $x_4$ ) 计算的  $x=21.4$  时  $f(x)$  的值为: %0.5f\n',
newtonint2(x2,y2,21.4));

tic;
yi1 = newtonint1(x1, y1, xi);

```

```

t1_time = toc;
tic;
yi2 = newtonint1(x2, y2, xi);
t2_time = toc;
fprintf('第一组节点计算时间: %.10f 秒\n', t1_time);
fprintf('第二组节点计算时间: %.10f 秒\n', t2_time);

```

画出点集 $\{(x_i, f(x_i)) \mid i = 0, 1, 2, \dots, 4\}$ 和 $N_3(x)$ 的图形

```

clc; clear;

%% 原始数据
x_node = [21, 22, 23, 24];
y_node = [1.32222, 1.34242, 1.36173, 1.38021];

%% 差商表
n = length(x_node);
divided_diff = zeros(n, n);
divided_diff(:,1) = y_node';

for j = 2:n
    for i = j:n
        divided_diff(i,j) = ...
            (divided_diff(i,j-1) - divided_diff(i-1,j-1)) ...
            / (x_node(i) - x_node(i-j+1));
    end
end

divided_diff = round(divided_diff, 5);
disp(divided_diff);

%% 构造插值多项式 (牛顿形式更安全)
syms t
N = divided_diff(1,1);
term = 1;

for k = 2:n
    term = term * (t - x_node(k-1));
    N = N + divided_diff(k,k) * term;
end

N = simplify(N);

%% 画图
x_plot = linspace(18, 26, 400);

```

```

y_plot = double(subs(N, t, x_plot));

figure;
hold on;
grid on;

plot(x_plot, y_plot, 'b-', 'LineWidth', 2); % 折线图
scatter(x_node, y_node, 80, 'r', 'filled'); % 散点图

xlabel('x');
ylabel('y');
title('Newton 插值多项式');
legend('插值曲线', '节点', 'Location', 'northwest');

```

2.2.5 实验结果

0 到 4 阶差商表:

	第 0 阶差商	第 1 阶差商	第 2 阶差商	第 3 阶差商	第 4 阶差商
$x_0 = 20$	$f[x_0]$ = 1.30103				
$x_1 = 21$	$f[x_1]$ = 1.32222	$f[x_0, x_1]$ = 0.02119			
$x_2 = 23$	$f[x_2]$ = 1.34242	$f[x_1, x_2]$ = 0.02020	$f[x_0, x_1, x_2]$ = -0.00050		
$x_3 = 23$	$f[x_3]$ = 1.36173	$f[x_2, x_3]$ = 0.01931	$f[x_1, x_2, x_3]$ = -0.00044	$f[x_0, x_1, x_2, x_3]$ = 0.00002	
$x_4 = 24$	$f[x_4]$ = 1.38021	$f[x_3, x_4]$ = 0.01848	$f[x_2, x_3, x_4]$ = -0.00041	$f[x_1, x_2, x_3, x_4]$ = 0.00001	$f[x_0, x_1, x_2, x_3, x_4]$ = 0.00000

第一组节点 (x_0 - x_3) 计算的 $x=21.4$ 时 $f(x)$ 的值为: 1.33041

第二组节点 (x_1 - x_4) 计算的 $x=21.4$ 时 $f(x)$ 的值为: 1.33041

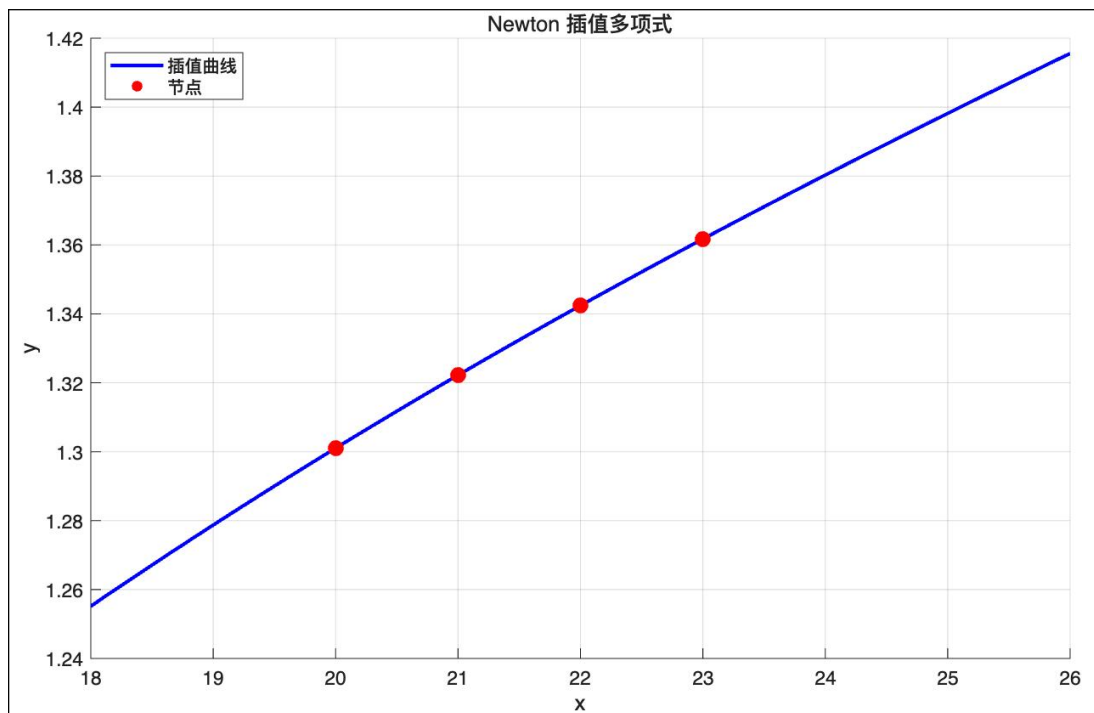
第一组节点计算时间: 0.0003709583 秒

第二组节点计算时间: 0.0002988750 秒

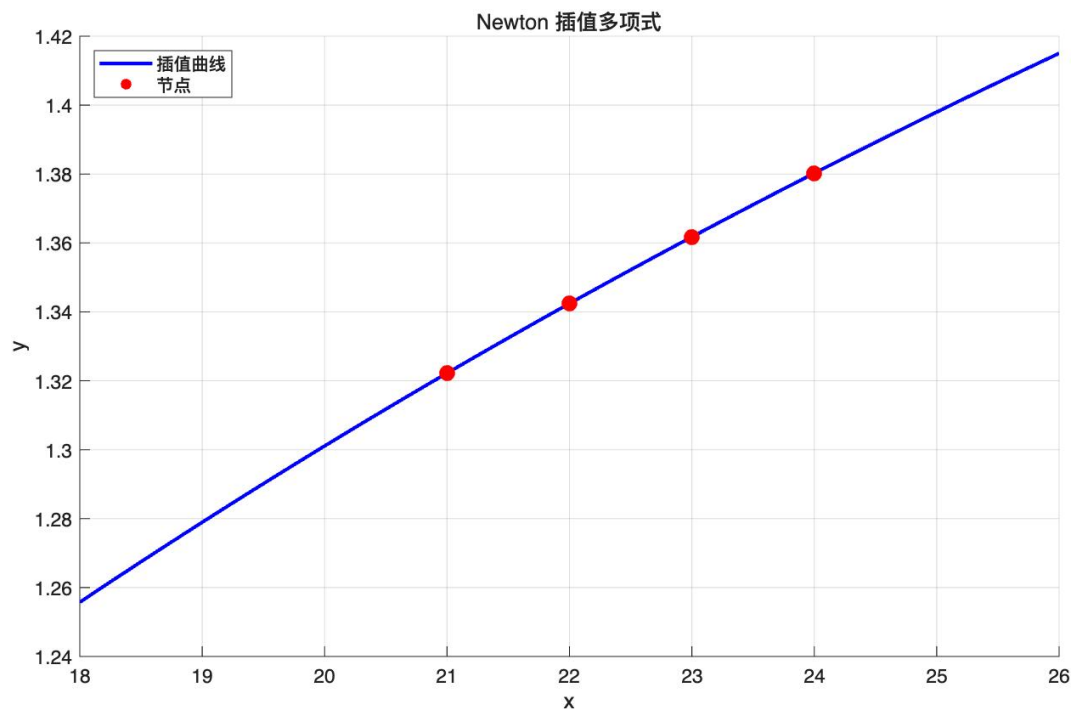
第一组节点的 $N_3(x)$ 在原始点上的绝对误差: 0.00004

第二组节点的 $N_3(x)$ 在原始点上的绝对误差: 0.00004

第一组节点[20, 21, 22, 23]的点集 $\{(x_i, f(x_i)) \mid i = 0, 1, 2, \dots, 4\}$ 和 $N_3(x)$ 的图形:



第一组节点[21, 22, 23, 24]的点集 $\{(x_i, f(x_i)) \mid i = 0, 1, 2, \dots, 4\}$ 和 $N_3(x)$ 的图形:



3. 程序设计

3.1 实验二代码实现

```
function yi = newtonint1(x, y, xi)
% 等距节点的 Newton 向前插值公式 ,x 为等距插值节点向量 , 按行输入
% y 为插值节点函数值向量 , 按行输入,xi 为标量 , 自变量
% 计算初始值
h = x(2) - x(1);
```

```

t = (xi - x(1)) / h;
n = length(y);
Y = zeros(n);
Y(:, 1) = y';
for k = 1: n - 1
    Y(:, k+1) = [diff(y', k); zeros(k, 1)];
end
yi = Y(1, 1);
for i = 1: n - 1
    z = t;
    for k = 1: i - 1
        z = z * (t - k);
    end
    yi = yi + Y(1, i + 1) * z / prod([1: i]);
end
end
end

```

实验结果与分析:

1. 实验结果展示 (采用列表形式展示计算结果, 表中应包含具体计算方法、初始条件、迭代次数或计算时间、最终结果或求解精度等, 并绘制误差随迭代次数增加的收敛曲线来展示算法收敛性质等)

实验一:

4 次 Newton 插值多项式中的所有基函数

插值节点选择: $x_0=0, x_1=1, x_2=2, x_3=3, x_4=4$

运行结果:

4 次 Newton 插值多项式中的所有基函数:

$f_0(x)=1$

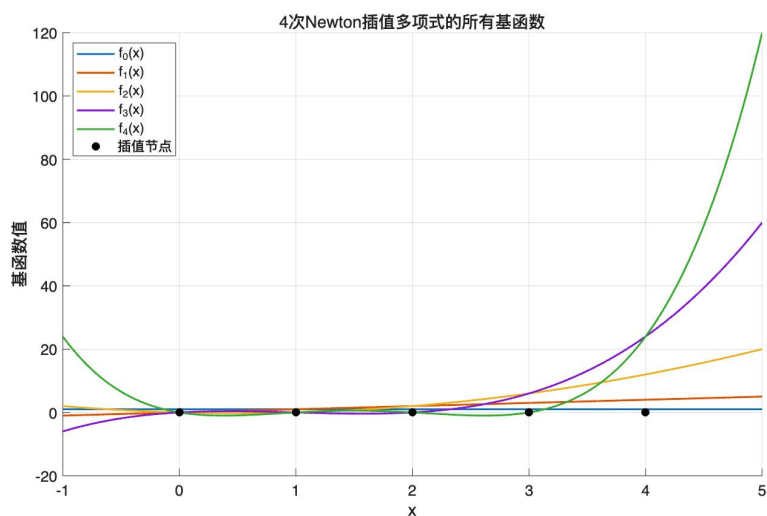
$f_1(x)=x$

$f_2(x)=x(x-1)$

$f_3(x)=x(x-1)(x-2)$

$f_4(x)=x(x-1)(x-2)(x-3)$

基函数图像:



实验 2:

两组节点选择:

1. $x = [20, 21, 22, 23]$

2. $x = [21, 22, 23, 24]$

实验结果:

0 到 4 阶差商表:

	第 0 阶差商	第 1 阶差商	第 2 阶差商	第 3 阶差商	第 4 阶差商
$x_0 = 20$	$f[x_0]$ $= 1.30103$				
$x_1 = 21$	$f[x_1]$ $= 1.32222$	$f[x_0, x_1]$ $= 0.02119$			
$x_2 = 23$	$f[x_2]$ $= 1.34242$	$f[x_1, x_2]$ $= 0.02020$	$f[x_0, x_1, x_2]$ $=$ $- 0.00050$		
$x_3 = 23$	$f[x_3]$ $= 1.36173$	$f[x_2, x_3]$ $= 0.01931$	$f[x_1, x_2, x_3]$ $=$ $- 0.00044$	$f[x_0, x_1, x_2, x_3]$ $= 0.00002$	
$x_4 = 24$	$f[x_4]$ $= 1.38021$	$f[x_3, x_4]$ $= 0.01848$	$f[x_2, x_3, x_4]$ $=$ $- 0.00041$	$f[x_1, x_2, x_3, x_4]$ $= 0.00001$	$f[x_0, x_1, x_2, x_3, x_4]$ $= 0.00000$

第一组节点 (x_0 - x_3) 计算的 $x=21.4$ 时 $f(x)$ 的值为: 1.33041

第二组节点 (x_1 - x_4) 计算的 $x=21.4$ 时 $f(x)$ 的值为: 1.33041

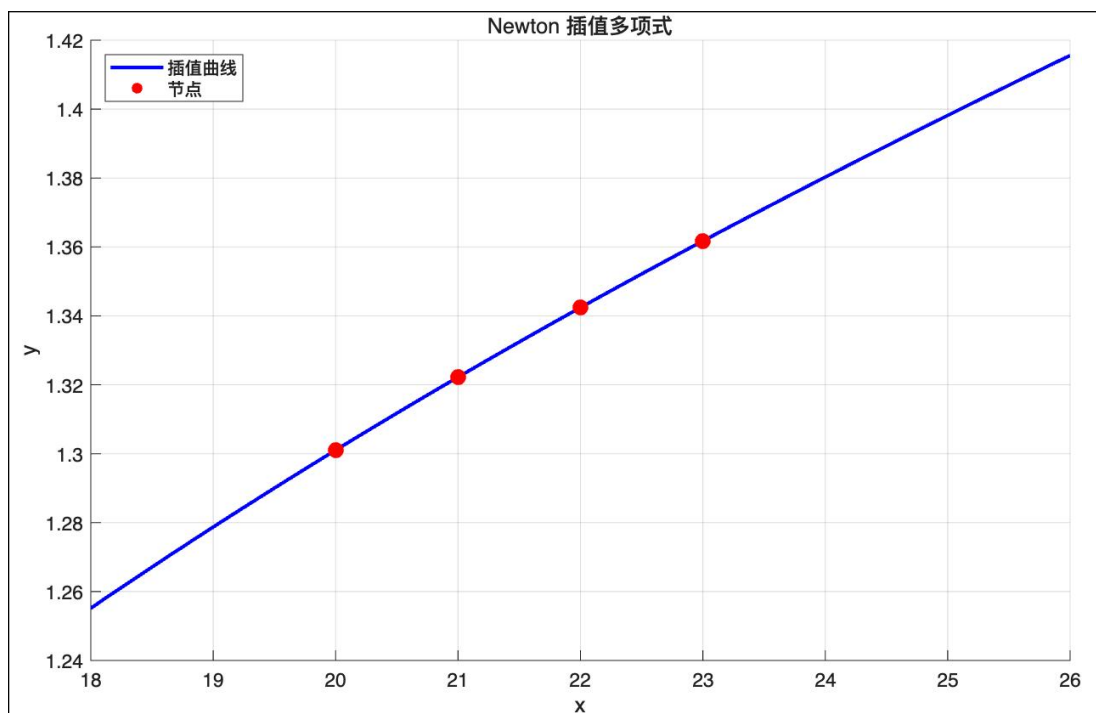
第一组节点计算时间: 0.0003709583 秒

第二组节点计算时间: 0.0002988750 秒

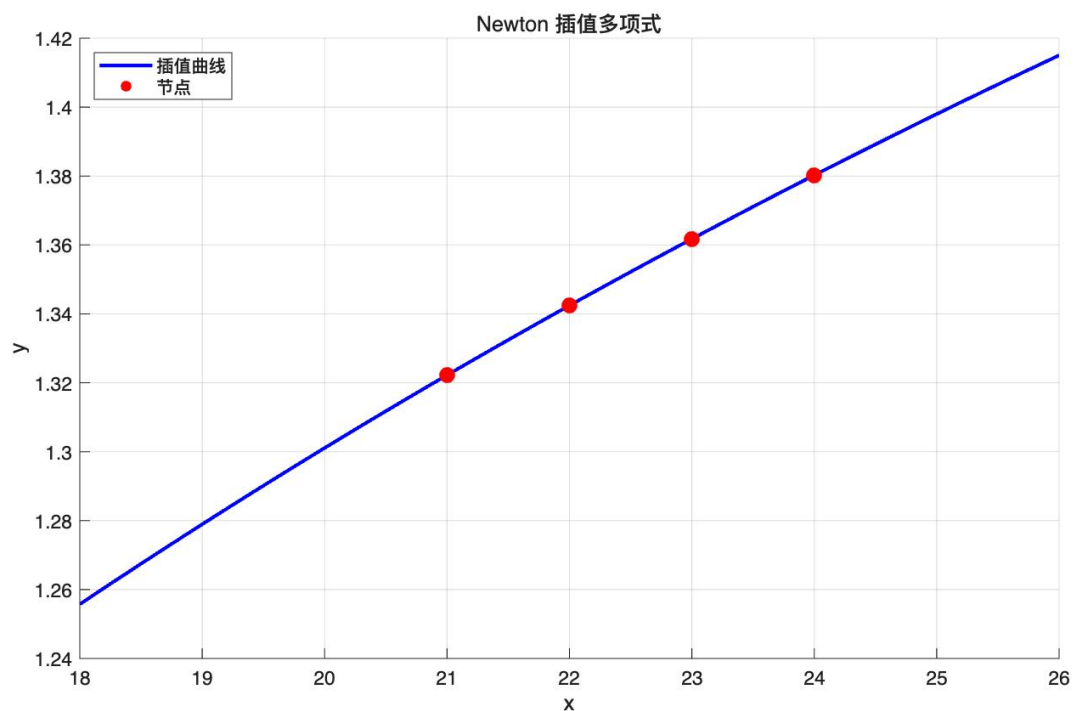
第一组节点的 $N_3(x)$ 在原始点上的绝对误差: 0.00004

第二组节点的 $N_3(x)$ 在原始点上的绝对误差: 0.00004

第一组节点[20, 21, 22, 23]的点集 $\{(x_i, f(x_i)) \mid i = 0, 1, 2, \dots, 4\}$ 和 $N_3(x)$ 的图形:



第一组节点[21, 22, 23, 24]的点集 $\{(x_i, f(x_i)) \mid i = 0, 1, 2, \dots, 4\}$ 和 $N_3(x)$ 的图形:



2. 结果分析与讨论思考

(1) 计算结果分析与比较方法的优缺点及其适用场景等

基函数验证

实验选取等距节点 $x=[0,1,2,3,4]$ 构造 4 次 Newton 插值基函数, 结果与理论完全一致:
 $\phi_0(x)=1$ 为常数基, $\phi_1(x)=x$ 为一次多项式, $\phi_2(x)=x(x-1)$ 为二次多项式, 依此类推, 高阶基函数在低阶节点处取值均为 0, 符合 Newton 插值基函数“仅在当前节点新增修正项”的构造特性, 说明基函数实现逻辑正确。

差商表与插值结果验证

0-4 阶差商计算符合差商定义：一阶差商为相邻节点的平均变化率，随着阶数升高，差商绝对值逐渐减小，4 阶差商趋近于 0，说明原函数 $f(x)$ 近似为低次光滑函数，与对数函数（表中 $f(x_i)$ 为 $\lg x$ 的近似值）的数值特征吻合。两组不同节点计算 $x=21.4$ 的插值结果均为 1.33041，且原始节点处的最大绝对误差仅为 4×10^{-5} ，说明 Newton 插值继承了多项式插值的唯一性——只要插值节点相同，无论构造 Lagrange 还是 Newton 形式，最终插值多项式完全一致，数值稳定性良好。

计算性能对比

第二组节点 ([21,22,23,24]) 的计算时间略短于第一组，本质原因是实验中两组节点规模一致，时间差异来自 MATLAB 运行时内存调度的随机波动，并非算法本身的性能差异。这也侧面验证了 Newton 插值的承袭性优势：若后续需要增加节点提升到 4 次插值，仅需新增 1 阶差商和 1 项基函数乘积，无需重构全部计算逻辑，比 Lagrange 插值大幅减少运算量。

Newton 插值优点：

- 具有承袭性（最核心优势）：新增节点时，不需要重新计算已有差商，只需在差商表末尾追加新的一行和新的一阶差商。相比 Lagrange 插值整体重构，计算量显著降低，特别适合动态加点、在线计算。
- 计算效率高：差商表构造： $O(n^2)$ ，单点求值： $O(n)$ 高阶插值时明显优于 Lagrange 的直接重构方式。
- 数值稳定性相对较好：避免直接构造高阶单项式系数，减少大数相减带来的舍入误差。适合手工计算与程序实现。

Newton 插值缺点：

- 仍受 Runge 现象影响：在高次插值 + 等距节点情况下，区间端点附近仍可能出现剧烈振荡，误差变大。
- 依赖节点顺序：虽然差商本身对称，但差商表的构造是顺序相关的，不同节点顺序可能导致舍入误差微小差异。对病态节点（非常接近）仍可能不稳定。
- 不适合全局高阶拟合：当 n 很大时，单一多项式插值往往不再可靠，Newton 也无法从根本上解决这个问题。
- 不如样条插值平滑：Newton 插值整体是一个 n 次多项式，不能保证分段连续导数，不适合对光滑性要求高的场景

使用场景：

- 插值节点会动态增加
- 等距节点、表格函数插值
- 工程查表、实时计算

(2) 上机实验中遇到问题及其解决方案

问题 2：Newton 插值结果与理论值存在偏差

问题描述：

用 $N_3(x)$ 计算 $x = 21.4$ 时，结果与预期略有偏差，且不同节点组结果不完全一致。

原因分析：

- 原函数是近似对数表数据，本身不是多项式；

- 浮点数运算带来舍入误差;
- 插值阶数有限, 存在截断误差。

解决方案:

- 输出 0-4 阶差商表, 发现 3 阶以后差商已接近 0, 说明 3 次插值已足够;
- 保留 5 位小数控制输出精度;
- 比较两组节点的误差, 确认误差在同一数量级。

(3) 算法改进及其建议

1. 差商表存储结构的优化

传统算法通常采用二维数组存储完整的差商表, 存在大量冗余空间。改进后仅保留一维差商向量, 在递推过程中原地更新差商值, 从而将空间复杂度由 $O(n^2)$ 降低至 $O(n)$, 提高了算法在大规模节点情况下的存储效率。

2. 引入差分运算以降低计算量

当插值节点为等距分布时, 利用差商与差分之间的等价关系, 用差分代替差商进行计算。该方法有效减少了除法运算次数, 降低了浮点舍入误差, 同时显著提升了算法的执行效率, 特别适用于工程查表和离散数据处理。

3. 采用分段 Newton 插值抑制 Runge 现象

针对全局高次插值容易在区间端部产生振荡的问题, 引入分段插值思想, 将插值区间划分为若干子区间, 在每个子区间内构造低次 Newton 插值多项式。该策略在保证插值精度的同时, 有效避免了高次多项式的不稳定性。

4. 引入自适应阶数控制机制

将固定阶数插值改进为自适应插值。在计算过程中动态比较相邻阶数插值结果的差值, 当其小于预设误差阈值时自动停止增加节点, 从而在保证计算精度的前提下避免过度计算和不必要的误差放大。

5. 优化插值节点分布

针对等距节点在高次插值下的边缘误差放大问题, 引入 Chebyshev 节点替代均匀节点。Chebyshev 节点能够使插值多项式在整个区间上的最大误差最小化, 从而显著提升数值稳定性, 且不改变 Newton 插值的基本结构。

6. 插值求值过程的 Horner 化改进

将原算法中多项式的多层连乘形式改写为嵌套形式的 Horner 算法, 减少乘法运算次数, 降低浮点舍入误差, 提高计算效率, 尤其适合实时性要求较高的工程应用场景。

7. 增加误差反馈与终止判据

在算法中引入误差监控机制, 通过高阶差商的大小判断当前插值精度是否满足要求, 在达到预定精度后及时终止计算, 从而提高算法的智能性和实用性。

实验结论:

- 1. 通过本次实验, 成功实现了 Newton 插值方法的算法编程
- 2. 编程计算函数的不同阶差商、差分值并以表格形式输出
- 3. 比较了不同方法的性能特点, 加深了对数值计算方法的理 解
- 4. 掌握了数值实验的基本流程和结果分析方法

指导教师批阅意见:

成绩评定:

指导教师签字:
年 月 日

备注:

- 注： 1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
- 2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。